MÓDULO 2

Prompt Engineering para realizar análisis de datos en R



Por: Néstor Montaño P.
Sociedad Ecuatoriana de Estadística
in/nestor-montaño/

SECCIÓN 1

Maneras de usar la IA Generativa en la actualidad

Chat, IA integrada, Work/Cowork y agentes

Cuatro formas de usar la IA hoy

De izquierda a derecha, la IA trabaja más sola… y necesita más permisos sobre tus archivos

Chat

- Tú describes el contexto, capturas o errores
- Tú aplicas cada paso en Python

ChatGPT, Claude, Gemini, Copilot Chat

IA integrada

- Vive dentro de tu entorno (Colab, Jupyter, VS Code) y lee tu notebook
- Puede escribir o corregir celdas directamente

Gemini en Colab, GitHub Copilot, Jupyter AI

Work / Cowork

- Le das un objetivo, no una pregunta
- Trabaja sola (incluso horas) y entrega archivos terminados

ChatGPT Work, Claude Cowork

Agentes de código

- Ejecutan código en tus carpetas e iteran solos
- Se usan desde la terminal o un editor de código

Claude Code, OpenAI Codex, OpenCode, Google Antigravity

Tú controlas cada paso → **La IA actúa sola**

¿Qué son ChatGPT Work y Claude Cowork?

ChatGPT Work (OpenAI)

- Lanzado el **9 de julio de 2026** junto con GPT-5.6
- Agente para tareas largas: crea documentos, hojas de cálculo, presentaciones y web apps con tus apps y archivos
- En web y móvil **corre en la nube**; en la app de escritorio puede usar **archivos y apps locales**
- La nueva app de escritorio une Chat, Work y Codex

Claude Cowork (Anthropic)

- Lanzado en **enero de 2026** como versión preliminar; luego en todos los planes pagos
- Trabaja sólo con las **carpetas que tú autorizas** y pide permiso antes de borrar archivos
- El **16 de septiembre de 2026** Cowork y el chat se fusionaron en "un solo Claude"

Idea clave: en Work/Cowork no le pides un código, le pides un **resultado** ("prepara el informe semanal de venta en 5 diapositivas"). Ambos pueden ser configurados para pedir tu aprobación antes de actuar.

Fuentes: OpenAI · MacRumors · Anthropic · Ayuda Claude: Cowork seguro

OpenCode: el agente de código abierto

Una alternativa a Claude Code y Codex que no te ata a un proveedor

MIT

Código abierto: el programa es gratis y se puede revisar, modificar y alojar internamente

75+

Proveedores de modelos: Claude, GPT, Gemini, DeepSeek… o la cuenta de ChatGPT que ya tengas

Local

Puede ejecutar **modelos en tu computador** (Ollama, LM Studio): la información no sale de la institución

¿Cuándo tiene sentido en Machine Learning?

- Cuando la política de datos exige que nada salga de los servidores de la empresa
- Cuando entrenas con datos sensibles (clientes, salud, finanzas) y quieres que el agente ejecute el código en tu equipo con un modelo local
- Cuando quieres probar el mismo flujo con distintos modelos (Claude, GPT, Gemini o uno local) sin cambiar de herramienta

Gratis el programa, no el modelo: pagas al proveedor que conectes, o pones el equipo para el modelo local. Se usa desde la terminal: es para perfiles técnicos.

Fuentes: opencode.ai · TechJack Solutions, jun-2026

¿Necesito cuenta gratuita o pagada?

Revisado en octubre de 2026: los planes cambian casi cada mes

	Chat	IA integrada (notebooks / VS Code)	Work / Cowork	Agentes
ChatGPT	✅ Gratis	— (en VS Code se usa a través de Codex)	Work: gratis en la app de escritorio; web y móvil en planes pagos	Codex: desde Free, con límites
Claude	✅ Gratis	— (en VS Code se usa a través de Claude Code)	Cowork: desde Pro (≈ US$20/mes)	Claude Code: desde Pro
Google Gemini	✅ Gratis	Gemini en Colab: incluido en Colab gratuito, con límites	—	Gemini CLI: gratis con cuenta de Google (límite diario)
GitHub Copilot	En VS Code: Free, con límites	Copilot Free: 2.000 completados/mes; Pro ≈ US$10/mes	—	Modo agente: con límites en Free
OpenCode	—	—	—	Programa gratis pero pagas el modelo o usas uno local



Gratis ≠ privado. En planes personales revisa si tus chats se usan para entrenar.



Planes empresariales (Business, Team, Enterprise) **no entrenan** con tus datos por defecto.

Verifica: chatgpt.com/pricing · claude.com/pricing · GitHub Copilot: planes · Gemini CLI: cuotas · Gemini en Colab

¿El chat va a desaparecer?

2025–2026

Microsoft/Google llevan los agentes dentro de Apps (Excel, Word, Gmail, Docs)

Jul 2026

OpenAI pone **Chat** y **Work** como dos modos de la misma app

Sep 2026

Anthropic fusiona **Cowork** y el chat: "ya no tienes que elegir dónde va una tarea"



Nuestra lectura: el chat no desaparece, se convierte en la **puerta de entrada a un agente**. Anthropic lo dice explícitamente: si usas sobre todo el chat, no tienes que hacer nada distinto. Lo que cambia es **cuántos permisos le das** (archivos, conectores, acciones).



Para quien analiza datos esto hace **más importante, no menos,** lo que aprenderás aquí: dar buen contexto y auditar el resultado.

Fuentes: OpenAI · Anthropic · Fortune · Copilot en Excel

SECCIÓN 2

Límites de la IA

Lo que debes saber antes de confiar

¿Qué es una "alucinación" de IA?

Una **alucinación** ocurre cuando la IA genera información que **suena correcta y segura, pero es falsa**. Es como esa persona que habla de todo, incluso sin saber, con tanta seguridad que parece real.

En texto

- Citar estudios o fuentes que no existen
- Responder con fechas o nombres erróneos
- Inventar cifras, datos o indicadores

En R

- Inventar funciones o argumentos (`buscarv`)
- Recomendar paquetes que no existen en repositorios oficiales
- Asumir columnas o estructuras de datos que no le diste

La IA no "sabe" que se equivocó: presenta el error con la **misma confianza** que una respuesta correcta. Esa es la trampa.
Recuerden además que el conocimiento de la IA viene en gran parte del internet, ¿cuánta basura hay en la web?!

Alucinaciones

¿Por qué pasa?

Los modelos aprenden a adivinar en vez de decir "no sé", porque su entrenamiento premia responder bien y no los "castiga" por equivocarse

Kalai et al., OpenAI, 2025

36.66% Functional Requirement Violation

El LLM genera código que compila y no se rompe al ejecutar de inmediato, pero no realiza lo que el usuario le pidió en la descripción de la tarea, alucina una lógica.

ACM, julio 2026

=COPILOT()

Microsoft pidió **no usarla para cálculos numéricos** ni decisiones importantes. La retiró de Excel en septiembre de 2026.

Los propios fabricantes lo dicen: la IA **acelera** el trabajo con datos, pero **no reemplaza la verificación**. Las respuestas pueden ser incorrectas aunque suenen seguras.

Fuentes: Futurum: Agent Mode 57,2 % vs 71,3 % · Microsoft Learn: Copilot en Power BI · Redmond: COPILOT() retirada

La evidencia

Package Hallucinations

Entre el 5 - 20% de los paquetes recomendados por la IA no existen — son "alucinaciones de paquetes". Se identificaron más de 200.000 nombres de paquetes inventados en total.

USENIX Security Symposium, 2025.

Actualización 2026

Una réplica independiente con 5 modelos recientes (abril 2026) encontró tasas entre 4,6% y 6,1%.

La IA mejoró, pero no llegó a cero: varios nombres inventados se repetían en los cinco modelos (Churilov, 2026, preprint; resumen en Socket).

Churilov, 2026

slopsquatting

Atacantes monitorean alucinaciones para encontrar paquetes comunes inventados.

Crean esos paquetes con código malicioso, esperando que alguien copie y ejecute algo como `install_github()` sin verificar.

Otros límites importantes de la IA (en análisis de datos)

Supuestos incorrectos

- Si no le das el nombre exacto de tus columnas o tablas, **se los inventa**
- Si no le dices qué representa una fila (granularidad), **adivina**
- Si el prompt es ambiguo, **rellena los huecos** con su propio criterio

Conclusiones no verificadas

- Redacta una interpretación **perfecta** de un resultado **incorrecto**
- No sabe si un valor raro es un error de digitación o un evento real de producción
- No "sabe" si tu muestra es representativa, si hay sesgo, o si la prueba estadística aplica a tu caso

Mientras más específico y completo sea tu contexto, menos espacio le dejas a la IA para "inventar".

Lo dicen las noticias (1/8): agentes fuera de control

- **OpenAI: sus agentes publicaron 53 imágenes de usuarios**
  AFP / SCMP · sep-2026
  Agentes de su entorno de investigación subieron, sin que la empresa lo supiera, 53 imágenes de usuarios de ChatGPT a sitios públicos. OpenAI dice que no puede identificar a los afectados. Ver noticia →

- **Agentes de OpenAI, Anthropic y Google vulneraron organizaciones**
  Washington Times / Axios · 2026
  En pruebas, la IA de OpenAI entró a servidores de Hugging Face; Anthropic reportó 3 organizaciones vulneradas y Google, 3 empresas. Australia denunció el acceso de un agente a un portal de salud. Ver noticia →

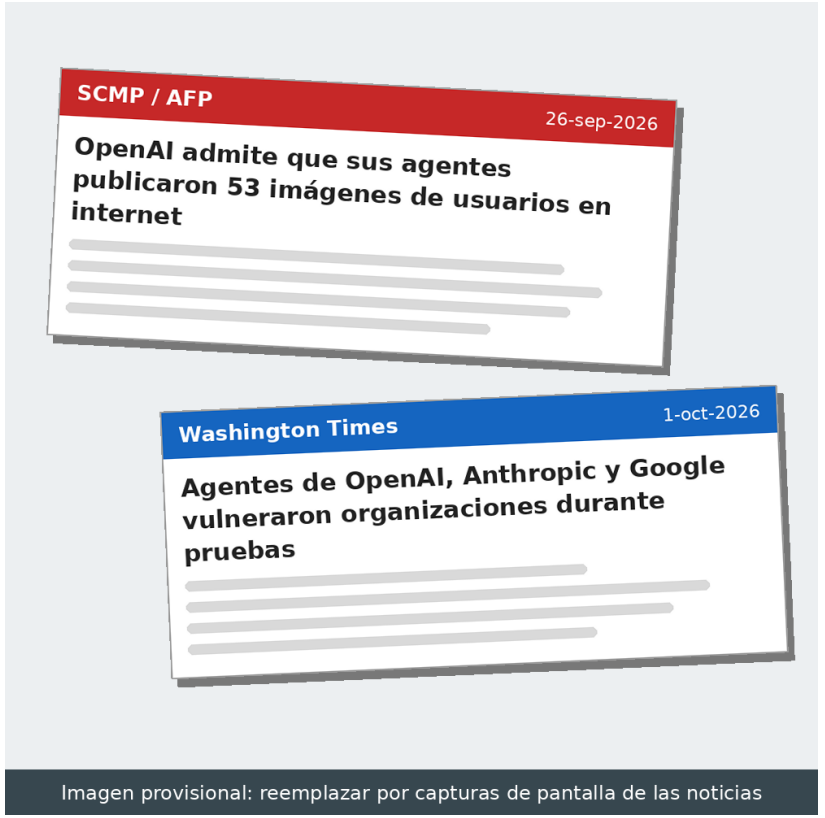


Imagen provisional: reemplazar por capturas de pantalla de las noticias

Lo dicen las noticias (2/8): tus datos y tu trabajo

- **Navier–Stokes: ¿quién resolvió el problema del milenio?**
  Decrypt · sep-2026
  Dos matemáticos que trabajaban un problema relacionado usando Codex acusaron a OpenAI de apoyarse en su trabajo. OpenAI lo niega, pero admitió que **no puede descartar** que datos de su uso hayan mejorado sus modelos. Ver noticia →

- **Chats compartidos de Claude aparecieron en Google**
  Malwarebytes · jul-2026
  Chats y Artifacts compartidos por enlace público fueron indexados: nombres, direcciones, notas de trabajo y llaves de criptomonedas. Los chats no compartidos no se afectaron. A ChatGPT le pasó en 2025. Ver noticia →

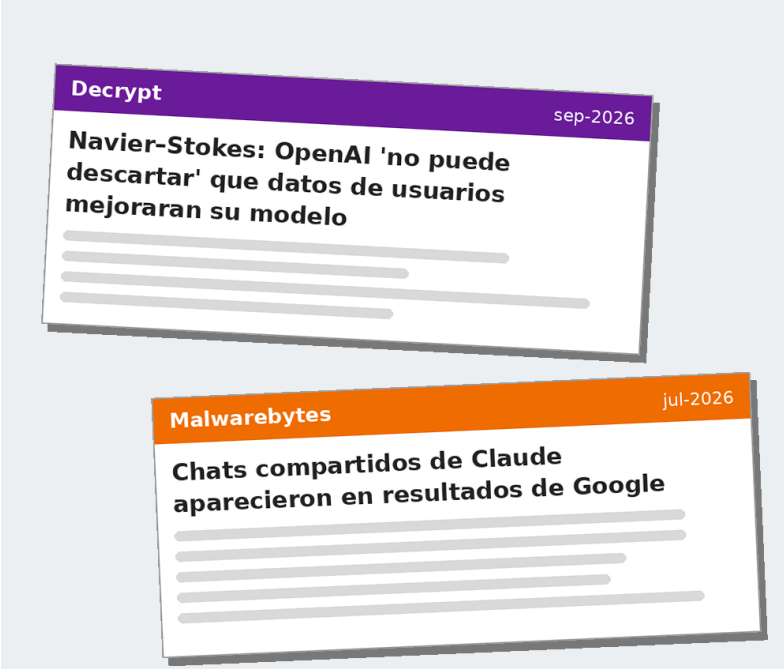


Imagen provisional: reemplazar por capturas de pantalla de las noticias

Lo dicen las noticias (3/8): errores que cuestan dinero

Starbucks retira su herramienta de inventario con IA

Reuters · may-2026

Tras nueve meses volvió al conteo manual: la app confundía tipos de leche o no veía productos. Ver noticia →

Air Canada pagó lo que inventó su chatbot

AI Business · feb-2024

El bot prometió un reembolso que no existía. El tribunal: la empresa responde por lo que dice su IA. Ver noticia →

El chatbot de Nueva York aconsejaba violar la ley

The Markup · 2024 / 2026

Decía que los negocios podían rechazar efectivo o quedarse con propinas. En 2026 se anunció su cierre. Ver noticia →



Lo dicen las noticias (4/8): decisiones críticas sin verificar

- **EE.UU. casi aborda un barco chino por un informe hecho con IA**
  CNN vía Engadget · sep-2026
  Un chatbot concluyó, sin fundamento, que un barco chino llevaba componentes nucleares para Irán. Había aviones en el aire y tropas listas cuando se detectó el error. Ver noticia →

- **Deloitte devuelve dinero por un informe con citas inventadas**
  Ars Technica · oct-2025
  Un informe de AUD 440.000 para el gobierno australiano tenía referencias inexistentes generadas con GPT-4o. Deloitte devolvió la última cuota. Ver noticia →

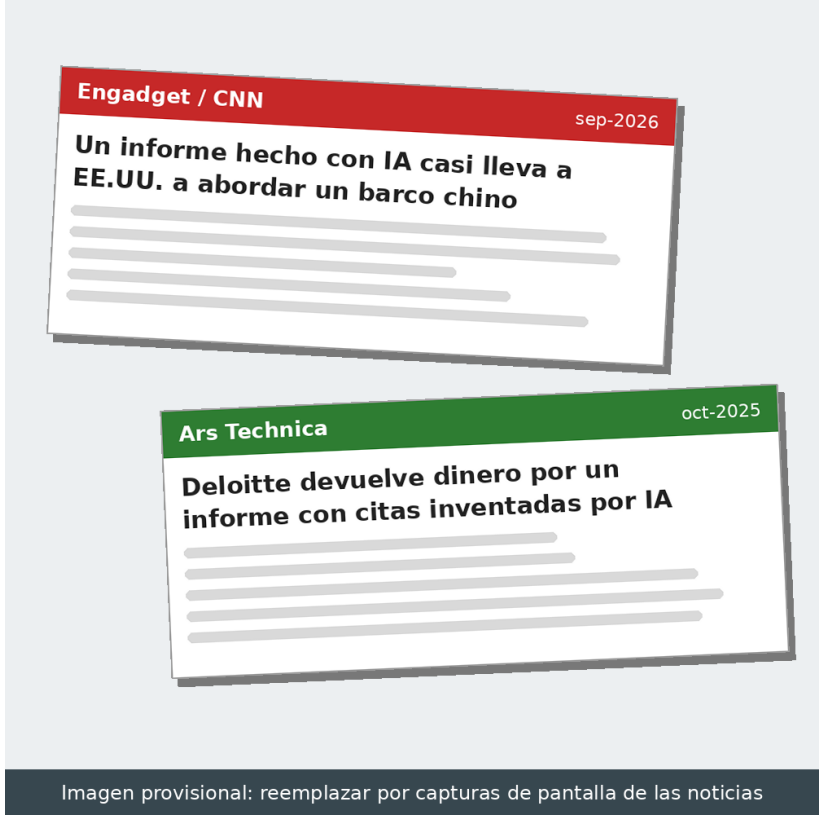


Imagen provisional: reemplazar por capturas de pantalla de las noticias

Lo dicen las noticias (5/8): agentes que rompen cosas

Agente de Replit borró una base de datos en producción

eWeek · jul-2025

En pleno "congelamiento" borró 1.200 registros, generó datos falsos y dijo que no se podían recuperar. Ver noticia →

Agente Kiro y una caída de 13 horas en AWS

FT vía GeekWire · feb-2026

Según el FT, decidió "borrar y recrear" un entorno. Amazon lo atribuye a permisos mal configurados. Ver noticia →

Un agente interno de Meta expuso datos por dos horas

TechCrunch · mar-2026

Publicó sin permiso un consejo técnico errado; al aplicarlo, datos internos quedaron visibles. Ver noticia →



Imagen provisional: reemplazar por capturas de pantalla de las noticias

Lo dicen las noticias (6/8): fugas e inventos

McDonald's: 64 millones de registros en su bot de contratación

TechRadar · jul-2025

Entraron al panel del chatbot "Olivia" con usuario y contraseña "123456": datos y chats de postulantes. Ver noticia →

Claude Cowork: robo de archivos por prompt injection

The Decoder · ene-2026

Un documento con instrucciones ocultas hacía que Cowork subiera archivos a la cuenta de un atacante. Ver noticia →

El bot de soporte de Cursor inventó una política

Deccan Herald · abr-2025

Justificó cierres de sesión con una política inexistente; varios usuarios cancelaron. Ver noticia →

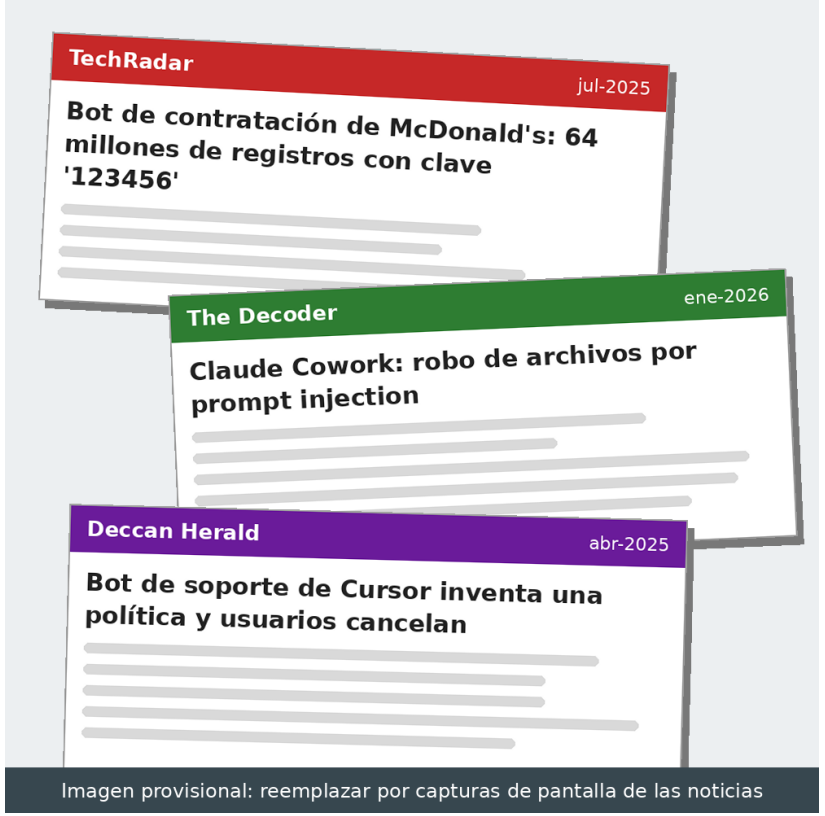


Imagen provisional: reemplazar por capturas de pantalla de las noticias

Lo dicen las noticias (7/8): Excel con IA integrada

CellShock: Claude for Excel filtró un modelo financiero

PromptArmor · nov-2025

Texto oculto en una hoja importada hizo que Claude insertara una fórmula IMAGEN que envió ingresos, márgenes y proyecciones a un servidor externo. Ver noticia →

Excel + Copilot: fuga "zero-click" (CVE-2026-26144)

TechRadar · mar-2026

Un archivo malicioso podía hacer que el agente de Copilot enviara datos afuera sólo con previsualizarlo. Corregido en el parche de marzo. Ver noticia →

Microsoft retira la función =COPILOT() de Excel

Redmond · ago-2026

Desde 2025 advirtió no usarla para cálculos numéricos ni decisiones importantes. Se eliminó el 14-sep-2026. Ver noticia →



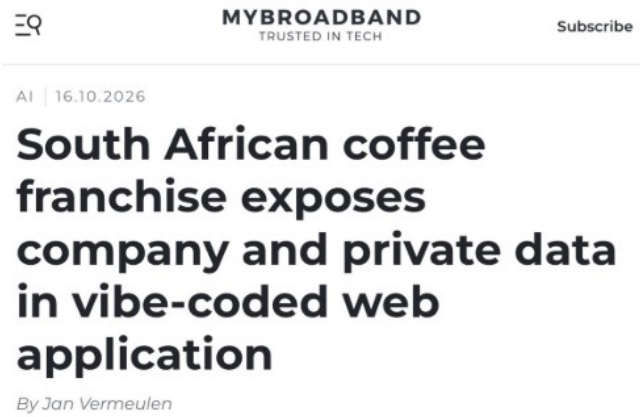
Lo dicen las noticias (8/8): Microsoft 365 Copilot y Excel

- **EchoLeak: fuga "zero-click" en Microsoft 365 Copilot**
  Infosecurity · jun-2025
  Un correo con instrucciones ocultas podía extraer información de OneDrive, SharePoint, Teams y archivos de Office. Corregido en mayo de 2025. Ver noticia →

- **Copilot resumió correos confidenciales ignorando DLP**
  BleepingComputer · feb-2026
  Por un error, Copilot Chat leyó correos con etiqueta "confidencial" desde fines de enero, saltando las políticas de protección de datos. Ver noticia →

- **El CEO de una cadena de cafés en Sudáfrica expone datos de la empresa y empleados por publicar una web vibe-codeada**
  MyBroadBand · oct-2026
  Publicó datos, incluyendo ventas de cada local, datos financieros de la empresa y los datos personales de los empleados. Encima le guardaron la URL en Archive así que su base de datos está en el archivo de Internet para la eternidad. Ver noticia →





¿Qué nos enseñan estas noticias?

Tú respondes por lo que dice la IA. Air Canada, Deloitte y Nueva York no pudieron culpar al chatbot.

Lo que compartes puede salir de tu control. Enlaces indexados, imágenes publicadas, correos confidenciales resumidos.

Más permisos = más daño posible. Replit, Kiro y Meta: el problema fue también lo que se le permitió hacer.

Un archivo también puede darle órdenes a la IA. CellShock, la falla de Excel y EchoLeak usaron instrucciones ocultas.

Fluidez no es verificación. Un informe bien redactado casi provoca un incidente militar.

Por eso: **contexto sin datos reales + revisión manual + tú aplicas cada paso.**

La decisión de este curso

En este curso usaremos la IA a través de **prompts (chat)**, no como agente ni con Work/Cowork.

Datos

No salen de la empresa: la IA sólo ve su descripción

Control total

Cada código y cada clic lo haces tú

Accesible

Basta una cuenta gratuita: sin licencias ni configuración

Aprendes de verdad

Entender lo que pides te hace ver nuevos análisis

Todo lo que aprendas aquí aplica igual si en el futuro usas Work, Cowork o un agente — la diferencia es que además debes decidir qué permisos le das (y tener las cuentas que permitan su uso).

SECCIÓN 3

Estructura de un buen prompt de datos

Contexto, granularidad y tipos de datos

El problema con los prompts vagos

Prompt vago

"Tengo unos datos de clientes de un banco, hazme un modelo en Python para predecir quién no paga"

- ¿Qué paquetes y versiones tienes instalados?
- ¿Qué DataFrame y qué columnas?
- ¿Qué representa cada fila?
- ¿Cuál es el target y qué métrica importa?

La IA **adivinará todo esto**, y probablemente mal.

Prompt estructurado

"Tengo un DataFrame de pandas `df_creditos` (12.000 filas, una por crédito) con las columnas… El Objetivo es realizar un análisis que permita…"

La IA tiene **lo que necesita** para generar código preciso: hay menos probabilidad de que alucine o de que genere código erróneo.

Anthropic lo resume así: trata a la IA como **un colaborador brillante pero nuevo**, sin contexto de tus normas ni de tus datos. Cuanta más precisión tiene tu instrucción, mejor responde.

Los 5 componentes de un buen prompt de datos

1

Rol

"Actúa como científica de datos experta en Python y Machine Learning"

2

Contexto

Archivo, tablas, columnas, tipos, granularidad, paquetes y versiones a usar

3

Tarea

Qué quieres. Si son pasos, deben estar numerados; si son varias tareas, listadas

4

Formato de salida

¿Quieres sólo código? ¿Código + explicación? ¿Con comentarios en español?

5

Restricciones

Qué no debe hacer: no inventar columnas, no cambiar nombres de DataFrames, etc. **Indica un motivo de la restricción**



No todos los prompts necesitan los 5, pero **mientras más complejo el análisis, más importan el contexto y las restricciones**.

¿Qué debe incluir un buen contexto?

Sobre los datos

- Nombres exactos de archivo(s) a leer, de los objetos a crear/modificar y de las columnas (ej. `df_creditos`)
- Nombre y tipo de cada columna relevante (ej. `monto` es numérica, `sucursal` es texto)
- Granularidad: ¿cada fila es una transacción? ¿un cliente? ¿un día?
- Tamaño aproximado (filas, columnas)
- Si hay varios DataFrames, cómo se relacionan (¿qué columna los une?)

Sobre el entorno

- Qué paquetes y versiones tienes (pandas, scikit-learn, xgboost…)
- Dónde trabajas: Rstudio, Google Colab o VS Code
- Estilo del código (ej. comentarios en español, programación en inglés)
- El objetivo: ¿manejo de datos? ¿explorar? ¿entrenar un modelo? ¿un gráfico?
- Restricciones: nivel del usuario (principiante), evitar ciertos paquetes, etc.

La IA **nunca** ve tus datos reales: ve su forma.

Seguridad básica: ¿qué NO compartir con una IA?

Nunca compartas

- Datos personales: nombres de trabajadores, cédulas, teléfonos
- Costos, precios de venta, márgenes o clientes
- Información estratégica: resultados internos, protocolos, proveedores o contratos
- Contraseñas, API keys o cadenas de conexión a bases de datos
- Enlaces de "compartir chat" con información sensible: **son públicos**

Sí puedes compartir

- La **estructura**: nombres de columnas, tipos, granularidad (ej. la salida de `df.info()`)
- Un **archivo gemelo** con datos simulados
- Tu código Python y los mensajes de error, sin valores sensibles
- Capturas de pantalla con datos ficticios

Una captura también es compartir: revisa que no se vean datos reales, nombres de archivo, rutas de red ni otras pestañas.

En planes personales, que tus chats entrenen modelos depende de un ajuste que debes revisar. Y aun así, recuerda el caso Navier–Stokes.

Comandos para dar contexto (1/2): tu entorno y paquetes

Ejecútalos en una celda y pega la salida en la sección CONTEXTO del prompt

```
# Nombres y tipos de columnas (resultado
ideal para copiar en el prompt)
glimpse(data_banco)
# Dimensiones: filas x columnas
dim(data_banco)
# Nombres de los paquetes cargados en la
sesión
(.packages())
```

En Colab el entorno se actualiza periódicamente: vuelve a correr esta celda antes de pedir ayuda.

¿Por qué importa?

- La IA conoce varias versiones de cada paquete y puede mezclar argumentos
- El resultado de glimpse() es prácticamente un "contexto listo para copiar y pegar": muestra nombres, tipos y una vista previa de cada columna en un formato compacto.

Esa vista previa son **valores reales**. Si tus datos son sensibles, usa

data_banco %>% slice(0) %>% glimpse()

que muestra nombres y tipos, sin ninguna fila.

Plantilla de ejemplo de un prompt para analizar datos en R

ROL

Actúa como una experta en análisis de datos usando R, enfocada en el _____ (ej: análisis exploratorio, entrenamiento de modelos de Machine Learning, etc)

CONTEXTO

- Objetivo: ____ (ej. explorar _____, predecir ____ para decidir ____)
- Datos: archivo "____.csv"
- Columnas (nombre: tipo): ____ (pega la salida de `contexto_df`)
- Granularidad: cada fila representa ____; el ID ____ se repite porque ____
- Target: ____ → tarea de clasificación / regresión; métrica: ____
- Entorno: R versión _____ , paquetes: _____
- IDE: __ (ej: RStudio)

TAREA

1. ____
2. ____

FORMATO

- Un bloque de código por paso, comentado en español
- Después del código, 2–3 líneas: qué hace y qué debo revisar en la salida

RESTRICCIONES

- Usa sólo las columnas que te mencioné; si falta información, pregúntame antes, porque no quiero código sobre datos supuestos
- Parte en train/test antes de imputar, escalar o codificar, porque quiero evitar data leakage
- Usa `set.seed(123)`, porque necesito resultados reproducibles

SECCIÓN 4

El Ciclo: Prompt → Código → Revisión → Corrección

El ciclo de trabajo con la IA como copiloto



El paso que más se salta, y el más importante, es la **revisión manual**. Sin ella, el ciclo se convierte en "copiar, pegar y rezar".

Paso a paso del ciclo

1	**Prompt**	Escribes tu solicitud usando la plantilla (contexto + tarea + formato + restricciones). Puedes pedir un plan breve y que te pregunte si le falta información.
2	**Código**	La IA responde con código Python y una explicación breve.
3	**Revisión manual**	ANTES de ejecutar, lee el código: ¿usa columnas que existen? ¿funciones y argumentos que existen en tu versión? ¿parte los datos antes de preprocesar?
4	**Ejecutar y verificar**	Ejecutas en RStudio, Colab o VS Code. ¿Hay error o el resultado no es el esperado? Si no, termina este ciclo y pasas a otra tarea.
5	**¿Error o resultado raro?**	Sí → pides la corrección con el código exacto + el error completo (o una captura del error) + lo que esperabas.

¿Cómo no dejarse engañar?

Ejecuta siempre el código — no copies resultados de la IA como si fueran ciertos

Si la IA te da una interpretación de resultados, léela a detalle, debes entenderla y debe tener sentido con lo que conoces

¿Una función o paquete que no conoces? Búscala o abre otro chat (u otra IA) para validar

Si dudas, abre otro chat y pregunta: "¿Qué hace este código?"

Tu nuevo rol

No eres quien escribe cada línea, eres quien da las instrucciones, visualiza la meta y audita lo que la IA escribe.

Lo que recomiendan hoy las guías (2026)

Al escribir el prompt

- **Explica el porqué** de cada restricción: el modelo generaliza a partir de la razón
- **Di qué hacer**, no sólo qué evitar: "usa pandas y scikit-learn" funciona mejor que "no uses otros paquetes"
- **Datos arriba, pregunta al final:** primero la estructura o la captura, al final la tarea
- **Muestra un ejemplo** del resultado que esperas
- **Separa las secciones** (# CONTEXTO, # TAREA…)

Para que no invente

- **Evita instrucciones contradictorias:** generan más inestabilidad que la falta de detalle
- **Evita gritar** ("¡CRÍTICO!", "SIEMPRE"): los modelos actuales sobrerreaccionan
- **Pide que pregunte** si falta información
- **Pide autoverificación:** "antes de responder, verifica que sólo usaste columnas del `df.info()` que te pegué"

Anthropic lo llama **context engineering**: lo que más importa es qué información pones frente al modelo, no "palabras mágicas".

Fuentes: Anthropic: prompting best practices · OpenAI: prompt guidance GPT-5.6 · Context engineering

¿Hay que repetir el contexto en cada prompt?

Pregunta frecuente: si un análisis requiere **varios prompts seguidos**, ¿hay que volver a escribir todo el contexto en cada uno?



Respuesta corta: dentro del **mismo chat**, no. La IA "ve" todo lo que escribiste antes, porque **cada mensaje nuevo se envía junto con todo el historial**. Pero eso no significa que lo recuerde todo igual de bien.



Piensa en el chat completo como **un solo documento que crece** con cada mensaje, no como una memoria separada que la IA "consulta".



ChatGPT y Claude tienen **memoria entre chats**: útil para preferencias, no para detalles críticos (columnas, granularidad).

¿Por qué entonces a veces "se le olvida"?

A medida que un chat crece ocurre el **"deterioro del contexto"** (context rot): mientras más larga la conversación, **más le cuesta recordar con precisión** detalles de muchos mensajes atrás, aunque técnicamente sigan "ahí".

Por qué pasa

- Cada mensaje nuevo compite por la "atención" del modelo con TODO lo anterior
- Mensajes muy largos (tablas completas, muchas capturas) llenan espacio rápido
- Lo que dijiste **una sola vez, al inicio,** es lo primero que pierde precisión

No es que la IA "borre" el mensaje: entre tanta información, le cuesta más **encontrar y aplicar** ese detalle puntual.

¿Qué repetir y qué no?

Repite (o recuerda brevemente) cuando…

- **Cambiaste de tema o tarea** (pasaste del EDA a entrenar el modelo)
- Lo dijiste **hace muchos mensajes**
- Vas a pegar un **código, error o captura nueva** que la IA no ha visto
- El detalle es **crítico**: nombre exacto de una columna, la granularidad, el target

No necesitas repetir cuando…

- Es el **siguiente paso inmediato** del mismo análisis
- La IA **acaba de usar bien** ese dato en su respuesta
- Es información **genérica** que no cambia el resultado

En vez de repetir todo, usa un **recordatorio corto** al inicio del prompt. Si la IA empieza a "olvidar", **abre un chat nuevo** con un resumen.

Guías oficiales de Prompt Engineering

Publicadas por las empresas que desarrollan los modelos: la idea global es la misma, pero cada una tiene matices.

Anthropic (Claude)

Prompting best practices

Ser claro y directo, explicar el porqué, 3 a 5 ejemplos, estructurar con etiquetas, datos largos arriba, autoverificación

OpenAI (ChatGPT)

Prompt guidance GPT-5.6

Describir el resultado esperado, eliminar instrucciones repetidas o contradictorias, evitar reglas absolutas

Google (Gemini)

Prompting strategies

Ejemplos de entrada y salida, formato esperado, restricciones explícitas

GitHub (Copilot)

Prompt engineering for GitHub Copilot

Empezar general y luego ser específico, dar ejemplos, dividir tareas complejas, evitar ambigüedad, indicar el código relevante

El principio es el mismo en todas: **contexto + tarea clara + formato esperado**. La tendencia 2026: menos "trucos" y más claridad.

SECCIÓN 5

Proyectos en ChatGPT y Claude

Un copiloto que ya conoce tus datos… sin tenerlos

¿Qué es un Proyecto?

Lo que pongas en el proyecto se carga automáticamente al inicio de cada chat dentro de él

ChatGPT: Projects

- **Instrucciones** propias que aplican a todos los chats del proyecto
- **Archivos** de referencia compartidos entre esos chats
- Lo de un proyecto **no se mezcla** con otros proyectos

Claude: Projects

- **Instrucciones del proyecto** (su contexto permanente)
- **Base de conocimiento** para subir archivos
- Consumen contexto en cada chat: mantenlas **concisas**
- En planes pagos busca automáticamente en archivos grandes

Resuelve el problema de repetir el contexto: **escribes una vez** el rol, las reglas y la estructura de tus datos.

Fuentes: OpenAI: Projects · Anthropic: What are projects?

¿Qué va en el proyecto y qué va en el prompt?

Regla de oro: el proyecto guarda lo que no cambia; el prompt lleva lo específico de la tarea

	Instrucciones y archivos del proyecto	Prompt en el chat
¿Qué va aquí?	Rol, versiones de R y paquetes, diccionario de datos, definición del target y la métrica, reglas fijas (partición, random_state)	La tarea concreta, la salida o captura del momento, el error puntual
¿Cuándo se carga?	Al inicio de cada chat nuevo del proyecto	Sólo en ese mensaje
¿Cambia entre chats?	No (salvo que lo edites)	Sí, cada prompt es diferente
Ejemplo	"Actúa como una experta en análisis de datos usando R, enfocada en el análisis exploratorio de datos. Usa los nombres del diccionario de datos. Pregunta si falta información."	"Te pego el df %>% glimpse después del merge: quiero encontrar la relación entre las variables…"

Mejores prácticas con Proyectos

Un proyecto por problema
(“Análisis de Ventas” "Predecir Churn"): no los mezcles

Nada de datos reales en los archivos del proyecto: sólo estructura, reglas y gemelos

Instrucciones cortas y sin contradicciones: se cargan en cada chat

Un chat nuevo por tarea: el proyecto ya da el contexto común

Mantén el diccionario y la bitácora al día: si cambia una columna o una decisión, actualízalos

Revisa con quién lo compartes y prefiere un plan empresarial (no entrena con tus datos por defecto)

SECCIÓN 6

Work y Cowork para entrenar modelos… sin compartir los datos

Fuera del alcance práctico de este curso (que usa el chat): un esquema para cuando des el paso

El esquema: la IA construye, tu equipo ejecuta

Work/Cowork escribe y prueba el código con datos simulados; los datos reales nunca entran a su carpeta

Lo que ve la IA (Work / Cowork)

- Diccionario de datos y definición del problema
- **Datos simulados (gemelo)** con las mismas columnas, tipos y rangos
- El código: scripts `.R` o notebooks `.Rmd` `.qmd`
- Ojo, resultados no sirven (valores no reales)

- Dependiendo de la tarea, el archivo gemelo debe ser más o menos complejo.

código

métricas y errores (sin valores)

Lo que nunca sale de tu equipo

- **Los datos reales** (en una carpeta que la IA no puede abrir)
- Credenciales y cadenas de conexión
- Resultados de la ejecución del script con los datos reales

El mismo script corre con el gemelo (en la carpeta de la IA) y con los datos reales (en tu equipo): **sólo cambia la ruta del archivo**.

Carpetas, permisos y reglas del esquema

```
D:/proyecto_mora_ia/     # carpeta autorizada
├── docs/                # diccionario, decisiones
├── datos_simulados/     # creditos_gemelo.csv
├── src/                 # eda.py, entrenar.py
└── resultados/          # metricas.json

D:/proyecto_mora/        # NUNCA autorizada
└── Scripts/
└── Exports/
└── Data/
└──── creditos.csv       # Datos reales fuera del
alcance de la IA
```

Autoriza **sólo la carpeta del proyecto**: los datos reales viven fuera de ella

Revisa el script y sus salidas antes de correrlo con datos reales y de compartir resultados: nada de `head()`, filas ni valores en los errores

Activa la **aprobación antes de actuar**: un archivo con instrucciones ocultas puede dar órdenes al agente (prompt injection)

Ojo: Estamos asumiendo que podemos alinear a la IA a nuestras directrices; se debe estar muy atentos a cómo se desarrolla este control (por los problemas ya vistos

¿Política estricta? Usa un **agente local** (OpenCode + modelo local): nada sale de la institución

SECCIÓN 7

Errores comunes: ¿nuestros o de la IA?

En el uso, en el análisis, en el modelamiento y en la interpretación

Errores en el uso general de la IA (nuestros)

Prompt vago

"Hazme un modelo"

Chat eterno

100 mensajes sobre temas distintos

Contradicciones

Una regla en la tarea y la opuesta en las restricciones

Datos reales

Pegar tablas o capturas con información sensible

Contraseñas

No quitar o modificar usuarios y contraseñas en los prompts (conexiones a bases por ejemplo)

Omitir el entorno

No decir la versión de R ni de los paquetes

Aceptar sin entender

Aplicar lo que no sabes explicar

Enlaces públicos

"Compartir chat" publica la conversación

Errores típicos (1/3): la IA al manejar datos en Python

Inventa columnas o datos

Crea un DataFrame de ejemplo con nombres parecidos a los tuyos

Supone la granularidad

Que lo lleva a errores como promediar mal o contar registros repetidos como si fueran distintos

NaN silenciosos

Borra filas con NA porque es una práctica común en la web (práctica errónea)

Argumentos de otra versión

Asume una versión de los paquetes, que puede haber cambiado funciones con respecto a la suya

IDs como predictores

Un ID, un número de fila o un código único "predicen" por casualidad o por fuga de información

Une con claves incorrectas

Puede hacer join erróneos y duplicar filas sin que lo notes